Module 02 — Modules and Packages

Phase: Advanced Python
Previous module: Functions, Decorators & Generators
Next module: OOP (Part 1 — Classes & Encapsulation)

Module Outcome:

- Import forms
- Module search path
- `if __name__ == "__main__"`
- Package structure and relative vs absolute imports
- pip / venv workflow
- Standard library highlights

What is a module? What is a package?

- A **module** is one Python file (`.py`) that holds related names: variables, functions, classes.
- A **package** is a **folder** of modules. An `__init__.py` file (even empty) tells Python: "this folder is importable."

Function → module → library (big picture):

Unit	Plain meaning
Function	A named group of statements
Module	A group of related functions/classes in one `.py` file
Package / library	A group of modules (folder or installable project)

Why do we need them?

Without modules, one script becomes thousands of lines — hard to read, hard to reuse, impossible to share. Real team codebases split work across many files. `import` lets you use that work without copy-paste.

Virtual environment (`venv`) is a private Python sandbox per project so Project A (library set X) and Project B (library set Y) do not fight on the same machine. Required on shared lab PCs and for clean deploy.

When you write `import math`, Python loads the standard-library `math` module and exposes its tools. When you `pip install requests`, that package lands in `site-packages` for the active environment.



Where used?
- Shared lab PCs — each project gets its own `venv` so library versions do not collide
- Team repos — `campus_utils` / `ksr_utils` packages imported by many scripts
- Capstone apps — split `main.py`, helpers, and `data/` instead of one giant file
- API clients — `pip install requests` (or similar) inside an activated venv
- Config / reports — `json`, `datetime`, `os` from the standard library with no extra install

Important Concepts

1. Import forms

Definition: Ways to bring another file's names into the current file.

Purpose: Reuse without duplication; keep call sites readable.

```
import math                          # whole module — use math.sqrt
print(math.sqrt(16))

from math import sqrt, pi            # specific names — shorter calls
print(sqrt(16))

import datetime as dt                 # alias — shorter module name
now = dt.datetime.now()

from os import path as osp           # alias for one imported name
```

Note:

- Avoid `from math import *` — star imports pollute the namespace and hide where names came from. Prefer `import math` or `from math import sqrt`.

Remember: Prefer `import math` or named imports — `from math import *` hides where names came from and invites clashes.

Print `ceil(4.2)` and `sqrt(81)` twice: once with `import math` (prefix) and once with `from math import ceil, sqrt` (no prefix).

```
import math
print(math.ceil(4.2))   # 5
print(math.sqrt(81))    # 9.0

from math import ceil, sqrt
print(ceil(4.2))        # 5 — no prefix
print(sqrt(81))         # 9.0
```

Expected Output:

```
5
9.0
5
9.0
```

Import `datetime as dt` and print today's date in a Hyderabad attendance header using `strftime('%d-%m-%Y')`.

```
import datetime as dt

today = dt.date.today()
print(f"Hyderabad Attendance — {today.strftime('%d-%m-%Y')}")
```

2. Module search path

Definition: The ordered list of folders Python checks when you `import`.
Purpose: Explains `ModuleNotFoundError` and why `venv` / project layout matter.

Python looks in:

1. Current directory
2. Directories in `PYTHONPATH`
3. Standard library
4. `site-packages` (pip installs)

```
import sys
print(sys.path)  # debugging import errors
```

Note:

- Wrong working directory → `ModuleNotFoundError` — run from the project root or fix `PYTHONPATH`.
- Typo in module name → `ModuleNotFoundError` — match the filename exactly (case-sensitive on Linux).

Print each `sys.path` entry with an index, and tag the current folder and `site-packages` entries.

```
import sys
import os

print("sys.path entries:")
for i, entry in enumerate(sys.path):
    tag = ""
    if entry == "" or entry == os.getcwd():
        tag = "  # ← current folder"
    elif "site-packages" in entry.replace("\\", "/"):
        tag = "  # ← site-packages (pip installs)"
    print(f"{i}: {entry!r}{tag}")
```

Save `namaste` in `greetings.py`, import it from `main.py`, and note the `ModuleNotFoundError` you get if that file is missing.

```
# Save as greetings.py (same folder), then run this as main.py
# --- greetings.py ---
# def namaste(name):
#     return f"Namaste, {name}!"

# --- main.py ---
from greetings import namaste

print(namaste("Priya"))  # Namaste, Priya!

# After rename/move greetings.py away and re-run:
# ModuleNotFoundError: No module named 'greetings'
```

3. `if __name__ == "__main__":`

Definition: A guard that runs a block **only when this file is executed as a script**, not when another file imports it.
Purpose: Put demos / CLI entry points here so `import greetings` does not print or run side effects.

```
# greetings.py
def namaste(name):
    return f"Namaste, {name}!"

if __name__ == "__main__":
    print(namaste("Priya"))   # runs only: python greetings.py
```

When you `import greetings` from `main.py`, `__name__` inside `greetings` is `"greetings"` — the demo block is skipped.

Note:

- Top-level `print` / `input` outside the guard runs on every import — keep importable modules quiet; put demos under `if __name__ == "__main__":`.

Common Error: Leaving `print` / `input` at module top level — every `import` triggers side effects. Guard demos with `if __name__ == "__main__":`.

Put `namaste` and a demo `print` under `if __name__ == "__main__":` in `greetings.py`. From `main.py`, import it and call `namaste` so the demo line does not run.

```
# save as greetings.py
def namaste(name):
    return f"Namaste, {name}!"

if __name__ == "__main__":
    print("Running as script:", namaste("Ravi"))
```

```
# save as main.py (same folder) — run: python main.py
import greetings

print(greetings.namaste("Priya"))
# No "Running as script" line — guard blocked the demo
```

Run `python greetings.py` so the `__main__` guard prints `Running as script: Namaste, Ravi!`.

```
python greetings.py
# prints: Running as script: Namaste, Ravi!
```

4. Package structure

Definition: A folder of modules you own (e.g. `ksr_utils/` / `campus_utils/`).
Purpose: Capstone and multi-file apps both expect this layout.

```
my_project/
├── venv/                # not committed to Git
├── requirements.txt
├── main.py
└── mypackage/
    ├── __init__.py      # package marker + optional exports
    ├── core.py
    └── helpers/
```

```
        ├── __init__.py
        └── strings.py
```

Note:

- Circular imports → `ImportError` / partial init — restructure; move shared code to a third module.
- Missing `__init__.py` (older Python / some tools) → package not recognized — add `__init__.py` in each package folder.

Create `campus_utils/formatters.py` with `format_inr` and import it from `main.py` as `from campus_utils.formatters import format_inr`. Print `format_inr(1250)`.

```
# Folder layout:
#   campus_utils/__init__.py   (can be empty for Program 1)
#   campus_utils/formatters.py
#   main.py

# --- campus_utils/formatters.py ---
def format_inr(n):
    return f"₹{n:,.2f}"

# --- main.py ---
from campus_utils.formatters import format_inr

print(format_inr(1250))  # ₹1,250.00
```

Expected Output:

```
₹1,250.00
```

Re-export `format_inr` from `campus_utils/__init__.py` so `main.py` can use `from campus_utils import format_inr`.

```
# --- campus_utils/formatters.py ---
def format_inr(n):
    return f"₹{n:,.2f}"

# --- campus_utils/__init__.py ---
from campus_utils.formatters import format_inr

__all__ = ["format_inr"]

# --- main.py ---
from campus_utils import format_inr  # short import via package init

print(format_inr(1250))  # ₹1,250.00
```

Expected Output:

```
₹1,250.00
```

5. Relative vs absolute imports

Definition:

- **Absolute** — full path from the top package: `from campus_utils.formatters import format_inr`
- **Relative** — path relative to the **current** package file: `from .formatters import format_inr` (`.` = this package)

Purpose: Inside a package, relative imports stay correct if you rename the top folder later; absolute imports are clearer from `main.py` outside the package.

```
# campus_utils/__init__.py  (inside the package — relative OK)
from .formatters import format_inr
from .validators import is_email

__all__ = ["format_inr", "is_email"]
```

```
# main.py  (outside the package — use absolute)
from campus_utils import format_inr
```

Note:

- Relative imports (`from .x import y`) only work **inside** a package — not in a loose script you run as `python formatters.py`.
- Prefer absolute imports from project entry points (`main.py`); use relative imports inside package modules / `__init__.py`.

In `campus_utils/__init__.py`, use a relative import (`from .formatters import format_inr`) and print `format_inr(99)` from `main.py`.

```
# campus_utils/formatters.py
def format_inr(n):
    return f"₹{n:,.2f}"

# campus_utils/__init__.py
from .formatters import format_inr

__all__ = ["format_inr"]

# main.py
from campus_utils import format_inr
print(format_inr(99))
```

In `campus_utils/report.py`, import `is_email` from `campus_utils.validators` with an absolute import and wrap it in `check(user)`.

```
# campus_utils/validators.py
def is_email(text):
    return "@" in text

# campus_utils/report.py
from campus_utils.validators import is_email  # absolute inside package also fine

def check(user):
    return is_email(user)
```

6. pip and venv workflow

Definition: A **venv** is a project-private copy of Python + installed libraries. **pip** installs packages from PyPI into the *active* environment.
Purpose: Shared lab PCs and two projects on one laptop need isolation — otherwise package versions collide.

```
python -m venv venv

# Activate (pick your OS):
source venv/bin/activate          # macOS / Linux
# venv\Scripts\activate           # Windows (Command Prompt)
# venv\Scripts\Activate.ps1       # Windows (PowerShell)

pip install requests
pip freeze > requirements.txt
pip install -r requirements.txt   # teammate / new machine
deactivate                        # leave the sandbox
```

Note:

- Committing `venv/` → huge repo and OS-specific paths — add `venv/` to `.gitignore`; commit `requirements.txt` instead.
- `pip install` globally (no venv) → version conflicts — always use a venv per project.

Interview: A module is one `.py` file; a package is a folder of modules (usually with `__init__.py`). `venv` isolates each project's pip installs.

Inside an activated venv with `requests` installed, import it and print `requests.__version__`.

```
# First (in terminal):
#   python -m venv venv
#   source venv/bin/activate          # Windows: venv\Scripts\activate
#   pip install requests
#   pip freeze > requirements.txt

import requests

print("requests version:", requests.__version__)
```

Call `requests.get` on `https://httpbin.org/get` (timeout 10) and print the status code and JSON `url`.

```
import requests

response = requests.get("https://httpbin.org/get", timeout=10)
print("Status:", response.status_code)
print("URL:", response.json()["url"])
# Deactivate venv first → ModuleNotFoundError: No module named 'requests'
```

If the lab PC has no internet, stop after Program 1 — that alone proves the venv works.

7. Standard library highlights

Definition: Modules that ship with Python — no `pip` needed.
Purpose: Prefer these before adding third-party packages for simple tasks.

Module	Purpose	Common functions
`math`	Numeric functions	`sqrt`, `ceil`, `floor`, `pow`
`datetime`	Dates and times	`datetime.now()`, `strftime`, `timedelta`
`random`	Pseudo-random	`randint`, `choice`, `shuffle`, `sample`
`json`	JSON ↔ Python	`loads`, `dumps` (strings); `load`, `dump` (files — homework)
`os`	OS interface	`getcwd`, `listdir`, `path.join`, `environ`

Note:

- Demo OTPs with `random` are fine in class — for real security use `secrets` later.
- Build paths with `os.path.join(...)`, not string `"folder/" + "file"` (Windows vs macOS).
- Prefer in-memory `json.dumps` / `json.loads` first. File `json.dump` / `json.load` is OK for the JSON file homework; deep `open` / CSV mastery comes in the Files module.

Use `random.choice` on a list of lunch spots and print today's pick.

```
import random

spots = ["Canteen", "Mess", "Tiffin centre", "Home"]
print("Today's lunch:", random.choice(spots))
```

Dump a small dict with `json.dumps`, print the JSON string, then `json.loads` it back and print the city.

```
import json

batch = {"city": "Pune", "batch": "PFS-01"}
text = json.dumps(batch)
print("JSON string:", text)

data = json.loads(text)
print("City:", data["city"])  # Pune
```

Expected Output:

```
JSON string: {"city": "Pune", "batch": "PFS-01"}
City: Pune
```

Step-by-Step Annotated Python 3 Code

Example 1 — math module

```
import math

radius = 7
```

```
area = math.pi * math.pow(radius, 2)
print(f"Area: {area:.2f}")

print(math.ceil(4.2))    # 5
print(math.floor(4.9))   # 4
print(math.sqrt(81))     # 9.0
```

Example 2 — datetime module

```
from datetime import datetime, timedelta

now = datetime.now()
print(now.strftime("%d-%m-%Y %H:%M"))   # Indian date format common in UIs

# 30 days from today (e.g., subscription expiry)
expiry = now + timedelta(days=30)
print("Expires on:", expiry.date())

# Parse string to datetime
dob = datetime.strptime("15-08-1995", "%d-%m-%Y")
print("Birth year:", dob.year)
```

Example 3 — random module

```
import random

# OTP for demo only — use secrets module in production
otp = random.randint(100000, 999999)
print("OTP:", otp)

colors = ["red", "green", "blue"]
print("Lucky color:", random.choice(colors))

nums = [10, 20, 30, 40]
random.shuffle(nums)
print("Shuffled:", nums)
```

Example 4 — json module (strings first; files below)

```
import json

# Python dict → JSON string
user = {"id": 1, "name": "Ananya", "city": "Chennai", "active": True}
json_str = json.dumps(user, indent=2)
print(json_str)

# JSON string → Python dict
parsed = json.loads(json_str)
print(parsed["city"])
```

Write/read a JSON file (JSON file homework; full Files deep-dive later):

```
import json

user = {"id": 1, "name": "Ananya", "city": "Chennai", "active": True}
```

```
with open("user.json", "w", encoding="utf-8") as f:
    json.dump(user, f, indent=2)

with open("user.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
    print(loaded)
```

Example 5 — os module

```
import os

print("Current folder:", os.getcwd())
print("Files here:", os.listdir("."))

# Cross-platform path (important for Windows + Mac labs)
data_path = os.path.join("data", "students.csv")
print("Data path:", data_path)

# Environment variable (common for config / secrets)
db_host = os.environ.get("DB_HOST", "localhost")
print("DB host:", db_host)
```

Example 6 — Creating and importing your own module

File: `greetings.py`

```
def say_hello(name):
    return f"Namaste, {name}!"

APP_VERSION = "1.0.0"

if __name__ == "__main__":
    print(say_hello("KSR Batch"))
```

File: `main.py`

```
import greetings

print(greetings.say_hello("KSR Batch"))
print("Version:", greetings.APP_VERSION)
```

Example 7 — Package with `__init__.py` + relative imports

File: `ksr_tools/__init__.py`

```
"""KSR training utility package."""
from .validators import is_email
from .formatters import format_inr

__all__ = ["is_email", "format_inr"]
```

File: `ksr_tools/validators.py`

```
def is_email(text):
    return "@" in text and "." in text.split("@")[-1]
```

File: `ksr_tools/formatters.py`

```
def format_inr(amount):
    return f"₹{amount:,.2f}"
```

File: `demo.py`

```
from ksr_tools import is_email, format_inr

print(is_email("student@ksr.com"))
print(format_inr(125000.5))
```

Expected Output:

```
True
₹125,000.50
```

Practice Exercises

Basic

1. Use `math` to compute EMI: `emi = P * r * (1+r)**n / ((1+r)**n - 1)`.
   Model answer:

```
P, annual_rate, years = 500000, 0.10, 5
r = annual_rate / 12
n = years * 12
emi = P * r * (1 + r) ** n / ((1 + r) ** n - 1)
print(round(emi, 2))
```

2. Print today's date in `DD/MM/YYYY` using `datetime`.
   Model answer:

```
from datetime import date
print(date.today().strftime("%d/%m/%Y"))
```

3. Generate a list of 6 unique lottery numbers (1–49) using `random`.
   Model answer:

```
import random
print(sorted(random.sample(range(1, 50), 6)))
```

4. Round-trip a dictionary with `json.dumps` / `json.loads` (no file).
   Model answer:

```
import json
d = {"city": "Hyderabad", "batch": "PFS-01"}
text = json.dumps(d)
print(json.loads(text)["city"])
```

5. Save and load a dictionary to `config.json` using `json.dump` / `json.load`.
   Model answer:

```
import json
cfg = {"theme": "light"}
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(cfg, f)
with open("config.json", "r", encoding="utf-8") as f:
    print(json.load(f))
```

Intermediate

1. Create module `date_utils.py` with `days_until(date_str)` and a `__main__` demo.
   Model answer:

```
# date_utils.py
from datetime import datetime, date

def days_until(date_str):
    target = datetime.strptime(date_str, "%d-%m-%Y").date()
    return (target - date.today()).days

if __name__ == "__main__":
    print(days_until("31-12-2026"))
```

2. Build package `inventory/` with `products.py` and `stock.py` (functions + dicts — no classes yet).
   Model answer:
   `inventory/__init__.py` (can re-export), `products.py` with `make_product(sku, name, price)`, `stock.py` with `add_stock(db, sku, qty)` using a dict as the store.
3. Write script listing all `.py` files in current directory using `os`.
   Model answer:

```
import os
print([f for f in os.listdir(".") if f.endswith(".py")])
```

4. Create venv, install `python-dotenv`, freeze to `requirements.txt`.
   Model answer:

```
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
pip install python-dotenv
pip freeze > requirements.txt
```

Interview-style

1. Difference between module and package?
   Model answer:
   A module is a single `.py` file you import. A package is a directory with `__init__.py` (or a namespace package) that groups related modules so you can write `from inventory.products import make_product`.
2. What is `if __name__ == "__main__":` used for?
   Model answer:
   It runs a block only when the file is executed as a script, not when imported. Put demos/CLI entry points there so

importing the module does not side-effect.

3. How does Python find modules when you `import requests`?
   Model answer:
   Python searches `sys.path` in order: the script directory, `PYTHONPATH`, site-packages (where pip installs packages), then standard library locations. The first matching package wins.
4. Absolute vs relative import — when use each?
   Model answer:
   Absolute (`from campus_utils.formatters import format_inr`) from entry scripts and for clarity. Relative (`from .formatters import format_inr`) inside package files / `__init__.py` so the package stays movable. Relative imports fail in a loose script run as `python formatters.py`.

Mini-Project (Assignment): Project Bootstrap Template

Title: `py_starter` — Reusable Project Skeleton
Lab kit (build here): 15_Lab_Assets/16_venv_package_bootstrap/
Also use: 30_ksr_utils_toolkit (optional polish)

What to build

Piece	Requirement
`venv/` + `requirements.txt`	Isolated env; freeze after installs
`campus_utils/` package	`__init__.py` + at least one helper module
`main.py`	Imports package helpers; prints a short demo
`data/sample.json`	Defaults loaded by a small `config` / `settings` module
`README.md`	Activate, install, run steps for a teammate

Stretch

- Add a second helper module (e.g. `validators.py`) and re-export it from `__init__.py` with a **relative** import.
- Pin versions in `requirements.txt` (`requests==…`).
- Document “works on Windows / macOS / Linux” activation commands.

Done when

- [ ] Starter kit / your `main.py` runs inside an activated venv
- [ ] Package import works without `sys.path` hacks
- [ ] README shows how a teammate recreates the env from `requirements.txt`
- [ ] Project pushed to GitHub (no `venv/` committed)

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why is `import json` preferred over writing your own JSON parser?
2. When would you use `os.path.join` instead of string concatenation?
3. What goes into `requirements.txt` and what should never go there?

4. How is a multi-file package folder similar to a reusable app unit in a larger project?
5. Why do we add `venv/` to `.gitignore` but commit `requirements.txt`?
6. What is `if __name__ == "__main__":` for?
7. Difference between `json.load` and `json.loads`?

Q1. Module vs package?
A: A module is a single `.py` file. A package is a directory of modules (with `__init__.py`) that can be imported as a hierarchy, e.g. `mypkg.submod`.

Q2. Purpose of `if __name__ == "__main__":`?
A: Code under this block runs only when the file is executed directly, not when imported. Enables reusable modules that also work as scripts.

Q3. Difference between `json.load` and `json.loads`?
A: `load` reads from a file object; `loads` parses a JSON string. (`dump` / `dumps` are the write-side pair.)

Q4. What is pip?
A: Python's package installer; downloads third-party libraries from PyPI into your environment's `site-packages`.

Q5. How do you share dependencies with team?
A: `pip freeze > requirements.txt`; teammates run `pip install -r requirements.txt` inside their venv.

Q6. Absolute vs relative import?
A: Absolute uses the full package path from the project root; relative uses `.` / `..` from the current package file. Use absolute from `main.py`; relative is common inside `__init__.py` and sibling modules.
